COSC 39: Theory of Computation

Credit Statement: Talked to Sair Shaikh'26

Problem 1. Binary addition.
Let's leave the theory land and apply our knowledge in finite automata to some real problems.
For any string $w \in \{0,1\}^*$, let $\langle w \rangle_2$ denote the integer represented by w in binary. For example:

$$\langle \epsilon \rangle_2 = 0, \quad \langle 0011 \rangle_2 = 3, \quad \langle 1010 \rangle_2 = 10, \quad \langle 1111 \rangle_2 = 15.$$

Let L and L' be arbitrary automatic languages over the alphabet $\{0,1\}$. Prove that the following language is also automatic:

$$\{w \in \{0,1\}^* : \langle w \rangle_2 = \langle x \rangle_2 + \langle y \rangle_2 \text{ for some } x \in L, y \in L'\}.$$

Hint: Try to prove first the case when both L and L' are languages containing a single binary integer.

Solution.

We can prove this by construction:

1. **Pre-processing (get rid of leading zeroes)** — For any binary string, the number of leading 0s does not change the integer value.

$$\langle w \rangle_2 = \langle 0^* w \rangle_2$$

Thus, we can replace the languages L and L' with the languages

$$L_p = \{w \in L \mid w = \epsilon \text{ or } w \text{ starts with } 1\}$$
$$L'_p = \{w \in L' \mid w = \epsilon \text{ or } w \text{ starts with } 1\}$$

This will not change the language L'' since for any word $x \in L$ there exists a word $x_p \in L_p$ such that $\langle x \rangle_2 = \langle x_p \rangle_2$. This also applies to L'_p. L_p and L'_p are both automatic. If we take their DFA's, we can simply remove the 0 transition from the start state, redirecting it into a fail trap.

2. **Reverse the DFAs** — As we have proven in class, for any automatic language, its reversal is also automatic. Thus, let D_r and D'_r be the DFAs representing the languages of the reversed binary strings in L_p and L'_p respectively.

- reverse all edges in L_p and L'_p
- swap the starting and accepting states
- convert the NFA to a DFA

An accepting path in D_r and D'_r now reads the bits of a word $x \in L_p$ and $y \in L'_p$ respectively from the least significant to most significant bit.

3. **Incrementally construct the NFA** — construct an NFA N with states being tuples (p, q, cb) where the state of D_r is p, the state of D'_r is q, and the carry bit is $cb \in \{0, 1\}$. The start state is $(p_0, q_0, 0)$ where p_0 and q_0 are the start states of D_r and D'_r respectively.

We can construct the transitions by following the rules of binary addition. For every transition $p \to_r p'$ in D_r and every transition $q \to_{r'} q'$ in D_r, make two transitions for each carry state.

$$(p, q, 0) \to (p', q', cb')$$
$$(p, q, 1) \to (p', q', cb')$$

This way, we can simulate memory by doubling our states to account for every possible carry bit value. However, we need to calculate the value of the edge and the value of the carry bit in the following state. To do this, we use basic binary addition rules.

Let r be the value of the edge $p \to p'$ and r' be the value of the edge $q \to q'$. Let cb be the value of the carry bit. Determine the edge value v and cb' by following these rules:

$$\text{for carry bit } 0:$$
$$(r = 0) + (r' = 0) \Rightarrow (v = 0) + (cb' = 0)$$
$$(r = 1) + (r' = 0) \Rightarrow (v = 1) + (cb' = 0)$$
$$(r = 0) + (r' = 1) \Rightarrow (v = 1) + (cb' = 0)$$
$$(r = 1) + (r' = 1) \Rightarrow (v = 0) + (cb' = 1)$$
$$\text{for carry bit } 1:$$
$$(r = 0) + (r' = 0) \Rightarrow (v = 1) + (cb' = 0)$$
$$(r = 1) + (r' = 0) \Rightarrow (v = 0) + (cb' = 1)$$
$$(r = 0) + (r' = 1) \Rightarrow (v = 0) + (cb' = 1)$$
$$(r = 1) + (r' = 1) \Rightarrow (v = 1) + (cb' = 1)$$

After constructing the NFA N, define the accepting states to be the states (p, q, cb) where both p and q are accepting in D_r and D'_r respectively and $cb = 0$. This is because the NFA N has to be composed of two binary strings valid in both L_p and L'_p. Additionally, the carry bit must be 0 as it indicates there is no more arithmetic needed.

For all the states (p, q, cb) where both p and q are accepting in D_r and D'_r respectively and $cb = 1$, create a transition with edge value of 1 to a **new** accepting state, effectively using up the carry bit and incorporating it into the NFA.

4. **Reverse again and convert to DFA** — right now, our NFA N is reading bits from least significant to most significant. We need to reverse this to read from most significant to least significant. Repeat these steps again:

- reverse all edges in N
- swap the starting and accepting states
- convert the NFA to a DFA

Now, we have a final DFA D_f that reads bits of w from most significant bit to the least significant bit.

Proof

We argue both directions of the equality $L(D_f) = L''$.

($\subseteq$) case: Suppose $w \in L(D_f)$. By construction of D_f as the DFA obtained from reversing the NFA N, there is an accepting path in N of the reverse of w—the bits read from least significant to most significant. By the definition of transitions of N, each step satisfies binary addition rules described above. Iterating through all these steps, starting from least significant bit, is binary addition. The acceptance state represents a tuple $(p_n, q_n, 0)$ where p_n and q_n are acceptance states in the DFAs L_p and L'_q respectively. Thus, by following the p and q states, we generate an accepting sequence of states in their respective DFAs, proving there exists some word $x \in L_p$ and some $y \in L'_p$ where $\langle w \rangle_2 = \langle x \rangle_2 + \langle y \rangle_2$. Thus, $w \in L''$.

($\supseteq$) Suppose $w \in L''$. There exist $x \in L$ and $y \in L'$ with $\langle x \rangle_2 + \langle y \rangle_2 = \langle w \rangle_2$. After pre-processing, there exist $x' \in L'_p$ and $y' \in L'_p$ with the same values. Thus x' follows an accepting path inside D_r and y' follows an accepting path inside D'_r. The transition function of N traces a path $(p_0, q_0, 0) \to \cdots \to (p_n, q_n, cb_n)$. Since $\langle x' \rangle_2 + \langle y' \rangle_2 = \langle w \rangle_2$, the final carry bit must be 0 and(p_n, q_n) are accepting states in D_r and D'_r respectively. Thereby, the path is accepting in N, so the reverse of $w \in L(N)$ and $w \in L(D_f)$.

Thus, $L(D) = L''$, and L'' is automatic.

Problem 2. When does an NFA accept everything?.
Let N be an arbitrary NFA with n states for some alphabet Σ of size at least 2. How long can the shortest word rejected by N be?
Put differently, let $L(N)$ be the language recognized by the NFA N, and define

$$\Sigma^{\leq f(n)} := \Sigma^0 \cup \cdots \cup \Sigma^{f(n)}.$$

What is the smallest $f(n)$ such that

$$\Sigma^{\leq f(n)} \subseteq L(N) \implies L(N) = \Sigma^*?$$

(a) Prove that $f(n) \leq 2^n$.

(b) Consider the alphabet

$$\Sigma := \left\{ \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \# \right\}.$$

Define s_n to be the following word:

$$s_n := \# \underbrace{\begin{bmatrix} 0 \\ 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} \cdots \begin{bmatrix} 0 \\ 1 \end{bmatrix}}_{n \text{ characters}} \# \begin{bmatrix} 0 \\ 0 \end{bmatrix} \cdots \begin{bmatrix} 1 \\ 0 \end{bmatrix} \# \cdots \# \begin{bmatrix} 1 \\ 1 \end{bmatrix} \cdots \begin{bmatrix} 0 \\ 1 \end{bmatrix} \#.$$

Notice that if we drop the brackets, then we can interpret the 0s and 1s between two consecutive # symbols (referred to as a *chunk*) as two binary numbers stacked on top of each other.

If we read the top numbers in every chunk from left to right, they form a binary counter from 0 to $2^n - 1$. For the bottom numbers, they form a binary counter from 1 to 2^n.

For example,

$$s_2 := \# \begin{bmatrix} 0 \\ 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} \# \begin{bmatrix} 0 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} \# \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} \#,$$

the top numbers are $00, 01, 10$, and the bottom numbers are $01, 10, 11$.

Construct an NFA recognizing the language $\Sigma^* \setminus \{s_n\}$.

Hint: You need to build separate gadgets to check that each chunk between two consecutive # has length exactly n, the two counts within a chunk are differed by one, the counts in adjacent chunks are consistent. For full credit, you can use at most $C \cdot n + o(n)$ states for constant C. Now you just proved $f(n) \geq \Omega(2^n/C)$.

$\star$ (c) Prove or disprove that $f(n) \geq 2^{n-o(n)}$.

Solution.

(a) Let N be an arbitrary NFA with n states for some alphabet Σ of size at least 2. Then we know that there exists some DFA D that is equivalent to this NFA, restricting to reachable states we also know that D has at most 2^n states.

Suppose $L(N) \neq \Sigma^\star$. Let w be the shortest word rejected by D. Thus there exists a walk through states $q_0, q_1, \ldots, q_r$ such that q_r is not an accepting state. Now note that since DFAs are deterministic, if a state repeats, i.e., $q_i = q_j$ for some $i \neq j$ then the (shorter) word obtained by deleting $w[i+1] \ldots w[j]$ also ends in a non-accepting state, contradicting our assumption. Hence, $|w|$ is at most the length of the longest path in D. Since we don't visit a state more than once, this length is at most the number of states in D minus 1 (starting state), i.e.,

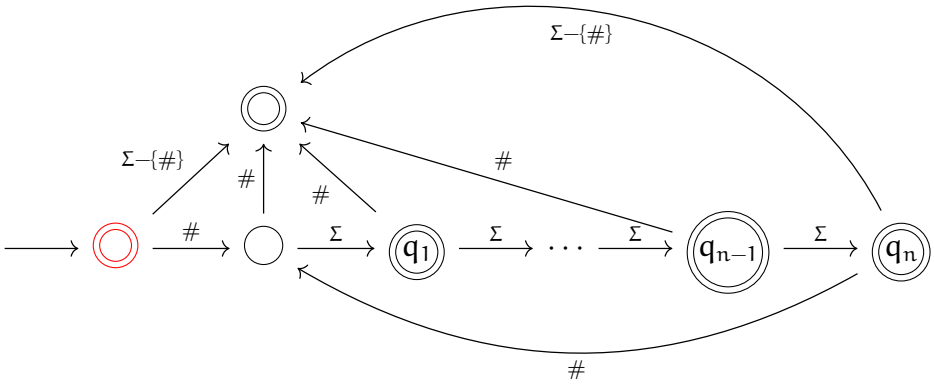
$$|w| \leq 2^n - 1$$

Thus, if all words of length $2^n - 1$ are accepted, then no words are rejected and $L(N) = \Sigma^\star$. Therefore,

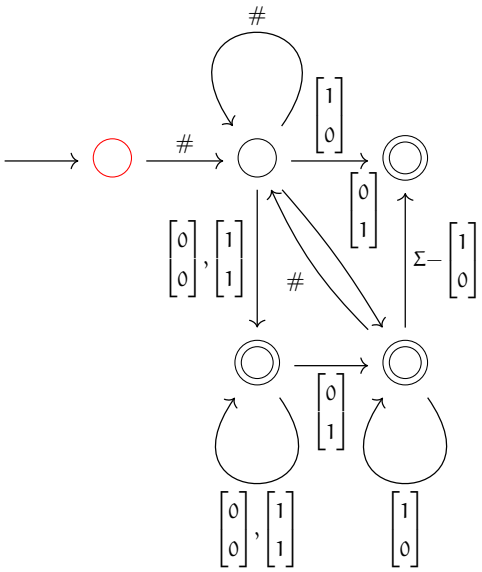
$$f(n) \leq 2^n - 1 \leq 2^n$$

(b) Our NFA N is the union of the following components. Each component is designed to accept if there is a violation of the structure of s_n.

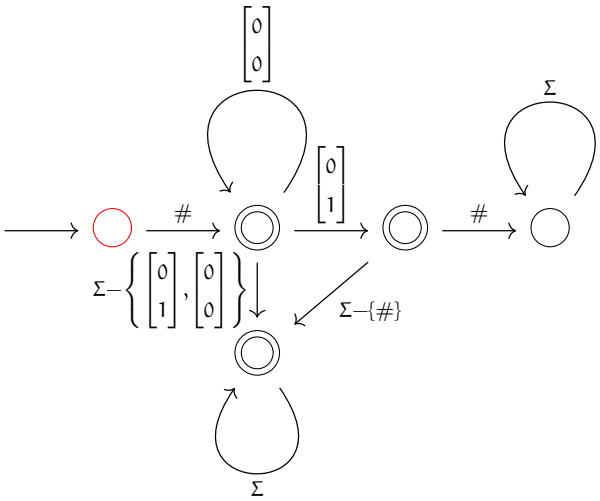
(i) checks if every chunk has length n.



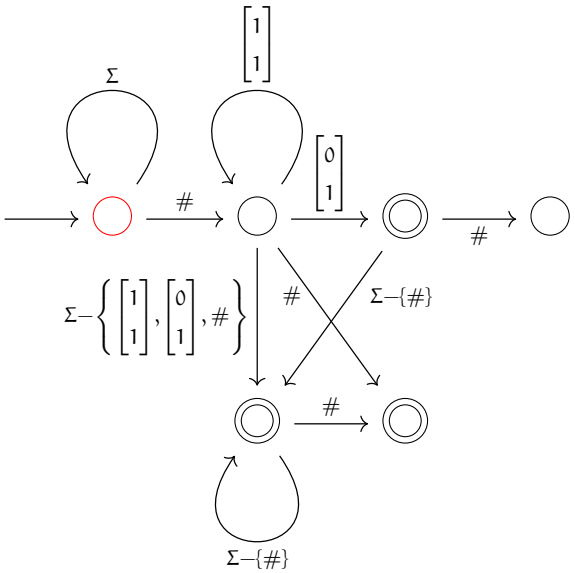
(ii) check if the two (top and bottom) counts within a chunk differ by one.



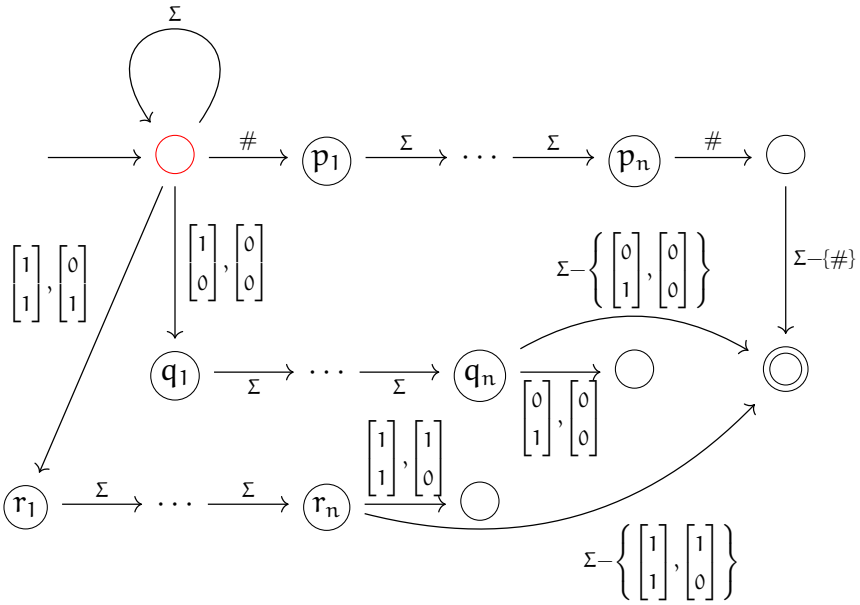
(iii) checks starting syntax.



(iv) checks ending syntax.



(v) compares characters that are n characters apart and thus checks if the bottom count in a chunk is consistent with the top count in the next chunk.



Note that the red states are starting states.

Let w be a string that is not accepted by N, then it must be rejected by all of the components. That is, the string has the following properties:

(a) It is not accepted by component (i), so every chunk has length exactly n.

(b) It is not accepted by component (ii), so the bottom number equals the top number $+1$ for all chunks.

(c) It is not accepted by component (iii), so it starts with a # has a series of $\begin{bmatrix}0\\0\end{bmatrix}$ followed by a single $\begin{bmatrix}0\\1\end{bmatrix}$ and a #. That is, the first chunk encodes 0 in the top row and 1 in the bottom row.

(d) It is not accepted by component (iv), so it ends with # followed by a series of $\begin{bmatrix}1\\1\end{bmatrix}$ followed by a single $\begin{bmatrix}0\\1\end{bmatrix}$ and a #. That is, the last chunk encodes $2^n - 2$ in the top row and $2^n - 1$ in the bottom row. It will also fail for strings that don't include # or have a final incomplete closure of #, but that is irrelevant as our other gadgets will accept these strings in these cases. This gadget only checks that the final # enclosure follows our defined format.

(e) It is not accepted by component (v), then for character a we have the following:

- If $a = \#$ then we have another # after n characters, or we terminate early.
- If the bottom entry of a is 0 the top entry of the character after n characters is 0, or we terminate early.
- If the bottom entry of a is 1 the top entry of the character after n characters is 1, or we terminate early.

That is the bottom chunk equals the top of the next chunk for every chunk.

These conditions uniquely determine the string s_n. Therefore, the only string not accepted by N is s_n, and hence $L(N) = \Sigma^\star - \{s_n\}$.